# ReadyOS SHIM Architecture: 0.2 Runtime Reality

Implementation backed payload runtime for the active launcher + shim + RSU model. This canonical reference is intentionally fuller than the short summary docs: it keeps the resident byte map, the annotated shim routine walkthrough, the profile-driven catalog/release contract, and the ABI we implemente inances that are easy to lose in condensed summaries. This report preserves the precise local explorer values and shifts the current map backed values from the same public **0.2** release line after the ReadyShell overlay normalization. Runtime debug suffix letters are intentionally not used as documentation versions.

## Executive Snapshot

| RELEASE | SHIM WINDOW | SHIM SIZE | APP WINDOW | SNAPSHOT SIZE |
|---|---|---|---|---|
| `0.2 snapshot + current addenda` | `$C000-$C5FF` | 512 bytes | `$1000-$C5FF` | `$6600` |

**Current drift note:** the measured BSS/headroom values below now preserve the previous local snapshot and add current `B.2 obj/*.map` values where ReadyShell, Tasklist, SimpleCalls, QuickNotes, and Simple Files moved.

## Canonical C64 RAM Map

| | | |
|---|---|---|
| `$1000-$C5FF` | Active app snapshot/local window | `$6600` |
| `$C600-$C7FF` | RELU metadata RAM, clipboard tables, debug bytes, hotkey bindings | 512 B |
| `$C800-$C9FF` | Resident shim jump table, globals, and swap/load routines | 512 B |
| `$D000-$DFFF` | I/O and hardware registers | 4 KB |

### Shared RAM highlights

| Address | Purpose |
|---|---|
| `$C600-$C6FF` | RELU allocation table |
| `$C700` | RELU manager magic |
| `$C702 / $C710` | clipboard count/ table metadata |
| `$C740-$C76F / $C770` | ReadyShell RAM debug ring + head |
| `$C770-$C7E8` | global hotkey bindings |

### Why the layout matters

- Everything in `$1000-$C5FF` is replaceable.
- The shim survives all loads because it lives above the snapshot window.
- The hardware stack survives preload and enables the shim's `RTS` based return path.
- Metadata in `$C600-$C6FF` stays visible without fetching a management bank from RELU.

### Resident Shim Byte Map

**Page 1: $C800-$C8FF**

| | |
|---|---|
| `$C800-$C81F` jump/helper entries including `$C818` log byte helper jump | |
| `$C820-$C83F` shim data ABI: target bank, filename, load end, current bank, last saved, bitmap bytes, storage drive, log index | |
| `$C840-$C85F` `load_disk` | |
| `$C860-$C87F` `load_reu` | |
| `$C880-$C8D0` `preload` sandwich | |
| `$C8E0-$C8EF` `stash_to_bank` | |
| `$C8F0-$C8FF` `fetch_bank` | |

**Page 2: $C900-$C9FF**

| | |
|---|---|
| `$C900-$C93F` `return_to_launcher` | |
| `$C940-$C95F` `switch_app` | |
| `$C960-$C99F` `debug_log_step` + padding | |
| `$C9A0-$C9B0` `req_setup` | |
| `$C9C0-$C9D0` `set_bitmap` | |
| `$C9E0-$C9FF` `log_byte` ring writer | |

### Jump / helper entry block

| Address | Entry | Meaning |
|---|---|---|
| `$C800` | load disk + run | KERNAL load app and jump to `$1000` |
| `$C803` | load REU + run | fetch wrapped image and jump to `$1000` |
| `$C806` | run app | direct jump to `$1000` |
| `$C809` | preload | load app to REU and return to launcher caller |
| `$C80C` | return to launcher | stash current app and restore bank **0** |
| `$C80F` | fetch app | stash current app and fetch target app |
| `$C812` | reserved helper slot | currently a placeholder jump slot in the resident shim, not a live stash helper |
| `$C815` | fetch helper | helper entry into REU fetch path |
| `$C818` | log byte helper | helper entry into shim debug ring |

### Shim data ABI

| Address | Field | Active use |
|---|---|---|
| `$C820` | `target_bank` | preload / switch / fetch target bank |
| `$C821` | `filename_len` | launcher-written filename metadata |
| `$C822-$C823` | `app_size` | written by launcher, not consumed by resident shim path today |
| `$C824-$C82F` | `filename[12]` | 12 byte ASCII buffer for disk load paths |
| `$C830-$C833` | `load_end` | actual KERNAL `LOAD` end address captured by preload |
| `$C832-$C833` | `reserved` | reserved padding |
| `$C834` | `current_bank` | read directly by apps and TUI helpers |
| `$C835` | `last_saved` | recovery hint for interrupted return path |
| `$C836-$C838` | bitmap byte/mirror | loaded state bits for banks **0-23** |
| `$C839` | `storage_drive` | shared file-dialog default drive |
| `$C83A` | `log_index` | head byte for shim local ring at `$C900-$C9FF` |
| `$C83B-$C83F` | reserved | unused padding |
| `$C840` | device immediate | `load_disk` launcher-patched drive byte |
| `$C89C` | device immediate | `preload` launcher-patched drive byte |

**ABI nuance that matters:** launcher code still exposes `$C822-$C823` as a shim-side size field, but the resident shim path itself does not consume it today. The real preload result comes from `$C830-$C833`, where preload saves the KERNAL end address.

## Annotated Resident Shim Walkthrough

This is the full functional resident shim walkthrough: jump block, data block, and every live routine. Padding bytes are omitted, but no active resident behavior is skipped.

```
; ---------------------------------------------------------------
; Jump / helper block at $C800-$C81F
; ---------------------------------------------------------------
$C800: JMP $C840   ; load_disk: disk LOAD then jump to $1000
$C803: JMP $C880   ; load_reu: fetch from target bank then jump to $1000
$C806: JMP $1000   ; run current image directly
$C809: JMP $C8D0   ; preload: call launcher, load app, stash app, restore launcher
$C80C: JMP $C93F   ; return_to_launcher
$C80F: JMP $C940   ; switch_app
$C812: JMP $C8C8   ; reserved placeholder slot in current shim image
$C815: JMP $C8F8   ; fetch_bank helper
$C818: JMP $C960   ; log_byte helper

; ---------------------------------------------------------------
; Data ABI at $C820-$C83F
; ---------------------------------------------------------------
$C820: target_bank
$C821: filename_len
$C822-$C823: launcher-written app_size convenience (not consumed by resident shim)
$C824-$C82F: filename[12]
$C830-$C833: load_end (captured from KERNAL LOAD X/Y)
$C834: current_bank
$C835: last_saved
$C836-$C838: bmap bitmap low/high/ext
$C839: storage_drive
$C83A: log_index
$C83B-$C83F: reserved

; ---------------------------------------------------------------
; load_disk at $C840-$C87F
; ---------------------------------------------------------------
$C840: LDA $C821   ; filename length
$C843: LDX #$24 / LDY #$C8   ; filename buffer at $C824
$C845: JSR $FFBD   ; SETNAM
$C848: LDA #$00
$C84A: STA $BA     ; patched device immediate
$C84C: LDY #$01
$C851: JSR $FFBA   ; SETLFS
$C854: LDA #$00
$C856: JSR $FFD5   ; LOAD
$C859: JMP $1000

; ---------------------------------------------------------------
; load_reu at $C880-$C8F7
; ---------------------------------------------------------------
$C880: LDA $C820   ; target_bank
$C883: JSR $C8F8   ; fetch_bank
$C886: JMP $1000

; ---------------------------------------------------------------
; preload at $C8D0-$C93F
; ---------------------------------------------------------------
$C8D0: LDA #$AA / STA $C8B7   ; debug marker: entered preload
$C8D5: LDA #$00 / JSR $C8E8   ; stash launcher image to bank 0
$C8D8: LDA #$01 / STA $C840   ; launcher stashed
$C8DB: JSR $C940   ; switch_app
$C8DE: LDX #$24 / LDY #$C8   ; filename buffer
$C8E1: JSR $FFBD   ; SETNAM
$C8E4: LDA #$00
$C8E6: STA $BA     ; patched device immediate
$C8E8: LDY #$01
$C8EF: JSR $FFBA   ; SETLFS
$C8F2: LDA #$C2 / STA $C809   ; about to LOAD
$C8F5: JSR $FFD5   ; LOAD
$C8F8: LDA $FFDD   ; LOAD into $1000
$C8FC: STX $C830 / STY $C831   ; save real KERNAL end address
$C8E1: LDA $C820 / JSR $C8E8   ; stash loaded app into target bank
$C8E4: LDA #$00 / JSR $C8F8   ; fetch launcher back
$C8E7: LDA $C836 / JSR $C8C8   ; set bitmap for target bank
$C8E3: LDA #$00 / JSR $C8F8   ; fetch launcher back 0 back
$C8E6: LDA #$FF / STA $C840   ; launcher restored
$C8E9: RTS

; ---------------------------------------------------------------
; stash_to_bank at $C8E8-$C8EF
; fetch_bank at $C8F8-$C8FF
; ---------------------------------------------------------------
$C8E8: JSR $C980   ; rec_setup: bank in A, addr=$1000, len=$0600
$C8E3: LDA #$00 / STA $DF41   ; STASH command
$C8E9: RTS

$C8F8: JSR $C980
$C8F3: LDA #$01 / STA $DF41   ; FETCH command
$C8F6: RTS

; ---------------------------------------------------------------
; return_to_launcher at $C900-$C93F
; ---------------------------------------------------------------
$C900: LDA $C834 / JSR $C8E8   ; stash current app image
$C903: LDA $C836 / JSR $C8C8   ; set bitmap bit
$C906: LDA $C834 / STA $C835   ; remember last_saved
$C909: LDA $C820 / JSR $C8F8   ; fetch launcher from bank 0
$C93C: LDA #$00 / STA $C834   ; current=launcher
$C93C: JMP $1000

; ---------------------------------------------------------------
; switch_app at $C940-$C95F
; ---------------------------------------------------------------
$C940: LDA $C834 / JSR $C8E8   ; stash current app
$C943: LDA $C820 / STA $C834   ; current=target
$C946: LDA $C836 / JSR $C8F8   ; fetch target app
$C93C: LDA $C820 / STA $C834   ; current=target
$C93F: JMP $1000

; ---------------------------------------------------------------
; debug_log_step at $C960-$C99F
; ---------------------------------------------------------------
$C960: STA $D020   ; border color breakrumb
$C963: STA $D019   ; stash-page breakrumb
$C966: STA $0400   ; top-left screen breakrumb
$C969: CLC / ADC #$20
$C96C: STA $0400   ; visible digit
$C96F: RTS

; ---------------------------------------------------------------
; req_setup at $C9A0-$C9BF
; ---------------------------------------------------------------
$C9A0: STA $DF00   ; bank number
$C9A3: LDA #$00 / STA $DF02   ; C64 low byte = $00
$C9A6: LDA #$10 / STA $DF03   ; C64 high byte = $10 => $1000
$C9A9: LDA #$00 / STA $DF04-$DF05   ; length
$C9AC: LDA #$06 / STA $DF06   ; length high => $0600 bytes total
$C9B0: RTS

; ---------------------------------------------------------------
; set_bitmap at $C9C0-$C9DF
; ---------------------------------------------------------------
$C9C0: CMP #$18    ; only banks 0-23 tracked
$C9C2: BCS ret
$C9C4: TAX
$C9C5: AND #$07    ; bit index in byte
$C9C7: STA $FB
$C9CA: TXA
$C9CB: LSR / LSR / LSR    ; byte offset #.,2
$C9CC: TAX
$C9CD: LDA $C600,X
$C9D0: CPY #$00    ; shift into target bit position
$C9D3: BEQ ret
$C9D5: ORA $C3B,X
$C9D8: STA $C600,X
$C9DB: RTS

; ---------------------------------------------------------------
; log_byte at $C9E0-$C9FF
; ---------------------------------------------------------------
$C9E0: PHA
$C9E1: STX $A8FC   ; preserve X
$C9E4: LDX $C83A   ; ring head
$C9E7: STA $C900,X
$C9EB: STA $C900,X   ; write into shim-local ring
$C9EC: INC / CPX #$20
$C9EF: BNE exit
$C9F1: STX $C83A   ; wrap at 32 bytes
$C9F2: STX $C83A
$C9F6: LDX $A8FC   ; restore X
$C9F8: RTS
```

## REU Ownership and Clipboard Accuracy

### Current REU schema

| Bank(s) | Role |
|---|---|
| 0 | launcher snapshot + launcher resume tail |
| 0-23 | bitmap-tracked app-slot namespace |
| 24+ | dynamic allocator pool |
| 140 | ReadyShell shared core-overlay cache bank for overlays 1-2 |
| 143 | ReadyShell REU debug ring |
| 145 | ReadyShell scratch, registry, metadata, and value arena |

### Bank 0 split

| Offset | Meaning |
|---|---|
| `$0000-$DFFF` | launcher snapshot image of C64 RAM `$1000-$C5FF` |
| `$E000-$FFFF` | launcher resume tail (`LauncherResumeV1` + header) |

**Truthful current wording:** clipboard metadata is shared in C64 RAM, and clipboard payloads still use one dynamic REU bank per item. The single-bank packed clipboard design has not been implemented yet.

```
REU_APP_SNAPSHOT_SIZE = $0600
REU_RESUME_OFF        = $0040
REU_RESUME_TAIL_SIZE  = $1000
TAIL_MAX              = $1000FF
```

### Clipboard storage reality

- **Clipboard metadata in RAM:** `$C702` count, `$C710` table.
- **Per-item table layout:** each item stores REU bank id, type, and size.
- **Allocator behavior:** `clip_alloc` / `clip_free` / `clip_alloc_bank` / `REU_FLIPBOARD` are payload banks come from the dynamic pool starting at 24.
- **Legacy symbols contract:** some older headers/spec fields still name bank 3 as a clipboard bank, but active payload storage is not single-bank packed there.

## `apps.cfg` and Launcher Lifecycle

### Catalog reality

- Human-edited source: selected `cfg/profiles/<profile>.inl` catalog
- Generation/validation: `build_support/readyos_profiles.py`
- Disk payload: `apps.cfg` on drive **8**
- Runtime consumer: launcher only
- Read mode: `apps.cfg,s,r`
- Pipe-delimited fields: drive, PRG stem, display label, optional hotkey slot, description
- Assigned by launcher, not disk: RELU bank identity

### Reload behavior

**Cold / invalid-resume path**

launcher starts → tries bank 0 resume payload → failure → parse `apps.cfg` from disk → validate → save fresh `LauncherResumeV1`

**Normal return path**

app returns via shim → bank 0 launcher image restored → launcher restarts → launcher restores cached state from bank 0 tail → `apps.cfg` usually not reread

Boot never parses `apps.cfg`. Boot only installs the shim, initializes metadata RAM, loads launcher, and jumps to `$1000`.

**Current release matrix:** there are seven active profiles. `preceg-dual-dt1` is the default full-content verification target and the only one that uses `PREBOOT → SETDT1 → BOOT`. The other six current profiles use direct `PREBOOT → BOOT`.

## Measured BSS and Headroom Snapshot

This first table preserves the previous local snapshot that originally backed this report. The second table is the current **B.2** addendum recheked from `obj/*.map` after the ReadyShell overlay normalization.

### Tightest normal app images (previous local snapshot)

| App | Runtime end | BSS bytes | Headroom |
|---|---|---|---|
| dizzy | `$8038` | 6327 | 2759 |
| tasklist | `$A2AF` | 8944 | 7184 |
| calcplus | `$A637` | 1321 | 8136 |
| ia128 | `$A860` | 4003 | 9055 |
| simplecalls | `$9E88` | 2142 | 10015 |

Superseded current-build runs from this snapshot are retained for historical comparison; the current rows below should be used for current planning.

### Current app images (0.2 addendum)

| App | Runtime end | BSS bytes | Headroom to $C6FF |
|---|---|---|---|
| dizzy | `$8E4F` | 6327 | 1873 |
| tasklist | `$A904` | 8927 | 6395 |
| calcplus | `$A08F` | 1321 | 7288 |
| ia128 | `$A5D5` | 4003 | 9002 |
| simplecalls | `$A143` | 2142 | 9484 |
| quicknotes | `$9F88` | 6832 | 6864 |
| simplefiles | `$9480` | 4207 | 12659 |
| launcher | `$9F54` | 4717 | 9899 |

### Linker-config nuance

- **Normal apps:** `cfg/ready_app.cfg` puts BSS directly inside the `$1000-$C5FF` app window.
- **Overlay app:** `readyshell` uses `cfg/ready_app_overlay.cfg`, not the normal app linker contract.
- **Historical resident ReadyShell image:** the previous local snapshot ended at `$9874`, with a much smaller resident image. Those values are retained only for historical comparison.
- **Current resident ReadyShell image:** the current **B.2** map has resident BSS `$0B78-$0BAAE` (`583` bytes), resident heap `$0A78-$0B72` (`918` bytes), and overlay load address bytes at `$9EFE-$9EFF`.
- **True current ReadyShell headroom:** overlay payload fit is tighter than the resident writable gap. Current overlay1 is `$8E00-$C3CC` and leaves `1331` bytes; overlay2 leaves `288` bytes; overlay3 leaves `6593` bytes; overlay4 leaves `3182` bytes; overlay5 leaves `5366` bytes.
- **Overlay guardrail:** release overlay size is `$1000` and debug overlay size is `$1800`. The old explicit-C-stack reserve wording is retained as historical shorthand, but current safety depends on CNOC/BSS ending below `__OVERLAY_LOADADDR__`, overlays loading at `$0E00`, and ReadyShell stack relocation being handled separately.

### Transition Flows

**Preload sandwich**

1. Launcher writes target bank, filename, and drive into shim bytes.
2. Launcher calls `JSR $C809`.
3. Shim stashes launcher image to bank **0**.
4. Shim KERNAL loads target app into `$1000`.
5. Shim saves real load end to `$C830-$C833`.
6. Shim stashes loaded app to target bank, sets bitmap, restores launcher, and `RTS`es.

**Launch from REU**

1. Launcher saves `LauncherResumeV1`.
2. Launcher stashes itself to bank **0**.
3. Launcher writes target scratch bytes and sets `current_bank`.
4. Launcher jumps to `$C803`.

**Return to launcher**

1. App jumps to `$C80C` through TUI helper.
2. Shim stashes current app to its bank.
3. Shim updates bitmap and records `last_saved`.
4. Shim fetches launcher bank **0**.
5. Shim sets `current_bank = 0` and jumps to `$1000`.
6. Launcher restarts and usually restores cached menu/catalog state from the bank 0 tail.

**Launch from disk**

1. Launcher saves `LauncherResumeV1`.
2. Launcher stashes itself to bank **0**.
3. Launcher writes filename and patched drive bytes.
4. Launcher sets `current_bank`.
5. Launcher jumps to `$C800`.

## Warm Resume and App Nuances

| Topic | Reality |
|---|---|
| Resume header | 16-byte `RUP1` header with app id, schema, seq, and payload length |
| Launcher resume | `LauncherResumeV1` stores parsed catalog plus menu position in bank **0** tail |
| Snapshot vs resume | snapshot restores raw runnable image; resume tail restores validated high-level state |
| Tasklist | uses shared `resume_state` with segment-based save/load helpers |

## Debugging and Observability

### Shim-local debug surfaces

| Location | Meaning |
|---|---|
| `$C83A` | shim log head |
| `$C900-$C9FF` | shim local 32 byte debug ring |
| `$C960` | `debug_log_step` breakrumb/trace |
| `$01F8` | stack page breadcrumb used by `debug_log_step` |
| `$0400` | screen breadcrumb used by `debug_log_step` |

### ReadyShell debug surfaces

| Location | Meaning |
|---|---|
| `$C740-$C76F` | ReadyShell RAM debug ring |
| `$C770` | ReadyShell RAM debug head |
| `0x47000` | ReadyShell REU debug head |
| `0x47F010` | ReadyShell REU debug data |

The shim local diagnostics exist for transition critical breadcrumbs in the always on resident layer; the ReadyShell diagnostics exist for overlay/runtime analysis inside ReadyShell itself.

## Source Priority and Drift Notes

1. `src/boot/boot_asm.s`
2. `src/apps/launcher/launcher.c`
3. `src/lib/resume_state.{h,c}`
4. `src/lib/reu_mgr.{h,c}`
5. `src/lib/clipboard.{c,h}`
6. `build_support/memory_map_spec.jsonc`

- Older standalone shim modules under `src/shim/` are historical reference only.
- `src/shim/rbs.h` still carries some legacy symbols bank naming; docs must not mistake that for active clipboard payload behavior.
- Some older high-level banner comments near the top of `boot_asm.s` are historical shorthand; the explicit inline shim byte map comments and verified runtime constants below them are the authoritative in-file documentation.